



Transcript

Week 12: Time Series Modelling

Video 1 - Time Series Modelling

Look around—weather apps, stock alerts, and Netflix recommendations all rely on time-based data.

Time series analysis focuses on data that changes over time, helping identify patterns, forecast trends, and make informed predictions. It applies to various fields, such as stock prices, weather patterns, and sales data. The analysis involves two key types of time series data: univariate, which tracks a single variable over time, and multivariate, which examines multiple variables and their relationships. Fundamental models like Autoregressive and Moving Average play a crucial role in analysing historical patterns to make future predictions. AR models use past values to forecast future trends, while MA models smooth out fluctuations for more stable forecasts. Understanding these concepts provides valuable insights for handling time-dependent data effectively.

How can time series insights enhance your predictions and decision-making? Let's dive in!

Video 2 - What is a Time Series?

Some data whispers. Time series data speaks in rhythms, patterns, and cycles. By learning to read its signals, you'll uncover hidden insights about what has happened—and what's likely to happen next. This is where data starts to tell a story worth listening to. Time Series Analysis is the study of data collected at regular time intervals. It helps identify trends, patterns, and seasonal changes in data over time. For example, tracking your daily steps over a month forms a time series—each day's count is one data point. By analyzing these points, we can understand how activity levels vary day by day. The line chart shows this time series, with days on the x-axis and steps on the y-axis, helping visualize daily changes clearly.

Time series analysis helps us understand how things change over time—and more importantly, it helps us predict what might happen next. By looking at past data, we can forecast future events. For example, retail stores use time series to plan for seasonal shopping trends, and financial analysts use it to monitor and predict stock prices. Time series plays a key role in helping businesses and industries make informed decisions. Time series data often shows specific patterns that help us understand what's happening and why. A **trend** is a long-term movement, like a steady increase or decrease in values. **Seasonality** refers to patterns that repeat at regular intervals—such as daily, weekly, or yearly cycles. **Cyclic patterns** are more irregular and can span longer periods, like economic growth or recessions. Finally, **noise** includes random changes in the data that don't follow a clear pattern. Spotting these characteristics is key to making accurate predictions.

This chart shows key characteristics of time series data. The red dotted line represents the **trend**, showing a steady increase over time. The green wavy line shows **seasonality**, a repeating weekly pattern. The blue dotted line represents **noise**, or random changes that don't follow a pattern. Understanding trend, seasonality, and noise helps us interpret data more clearly and make better forecasts. Here, we're looking at two types of time series data.

First, consider univariate time series. This type of data tracks a single variable over time.



The chart on the left shows an example where only one variable – temperature – is recorded daily over 30 days. Univariate time series helps identify trends, seasonal patterns, or anomalies in that single variable, like monthly sales with a seasonal spike in December.

On the right, we have multivariate time series. This type includes two or more variables tracked simultaneously. The example here shows temperature, humidity, and wind speed tracked together over time. Using multivariate data allows us to explore the relationships between these different variables, revealing how they might influence each other, giving us a more complete view of changing conditions. For instance, we can see how temperature and humidity might relate to energy use.

Now that you know the difference, think about a dataset you've worked with—was it univariate or multivariate? Understanding the type of time series you're working with is the first step to choosing the right tools and building better models.

Video 3 - Introduction to Feature Engineering in Time Series Data

Behind every powerful prediction is smart feature engineering. It's how we turn raw time series data into signals that actually mean something. By shaping the data—like adding time lags or capturing seasonality—we give models what they need to make accurate forecasts. It's not just data prep; it's where the magic starts.

In time series analysis, feature engineering helps us turn raw data into something more useful. It allows us to highlight important patterns—like seasonality or trends—and prepare the data for more accurate predictions. But time series data has its own challenges. Time dependency, seasonality, noise, and irregular intervals can make this process tricky. Understanding these challenges is the first step in building smarter models.

When working with time series data, trends can make forecasting challenging because they cause the data's statistical properties to change over time. To overcome this, we use detrending techniques, such as differencing, where we subtract each data point from the previous one. For instance, if a series has values like 5, 7, 10, and 12, taking the difference transforms it into 2, 3, and 2. This removes the trend, stabilizing the data. As the graph demonstrates, removing trends results in stationary data, making predictions more reliable and accurate.

This visual compares a time series with a clear upward trend and its detrended version. When we remove the trend, the data becomes more stable and easier to interpret. In this case, detrending was done by subtracting the average value from each point. This reveals the true fluctuations in the data—giving us a clearer view of what's really happening over time.

Seasonality is a common feature in time series data, where patterns repeat at regular intervals—like weekly shopping habits or yearly climate shifts. These repeating patterns can make it hard to see what else is going on in the data. To tackle this, we use a method called decomposition. It breaks the data into three parts: the trend, the seasonal pattern, and the residual, or what's left over. This helps us study each part separately and understand what's really driving the changes.

This chart breaks down the original time series into three components: trend, seasonality, and noise. By separating these elements, we can clearly see repeating patterns, long-term changes, and random fluctuations. This decomposition helps us better understand the data and build more accurate models for forecasting.

Noise in time series data can hide important patterns, making it harder to analyse and predict trends. One simple way to manage this is by using a moving average, which



smooths out random spikes. For example, a three-day moving average takes each group of three values, finds their average, and replaces the middle value with it. This helps reveal the overall trend more clearly by reducing short-term fluctuations. This chart compares the original time series data with a smoothed version using a three-day moving average. Smoothing helps reduce random fluctuations—also known as noise—making it easier to spot overall trends. In the orange line, you can see how the moving average softens the sharp ups and downs, giving us a clearer view of the general movement in the data. This technique is often used in time series analysis to improve accuracy and pattern recognition.

Missing values can cause problems in time series data by breaking the flow and reducing accuracy. One simple solution is forward filling, where each missing value is replaced by the last known value. For example, in the series five, Not-a-Number, seven, eight—forward filling gives five, five, seven, eight. This keeps the data consistent and helps models perform better.

This chart shows how missing values can be handled in time series data. The grey line marks where values are missing. Using forward fill, we replace each missing point with the last known value. The green line shows the result—continuous, complete data that is easier to analyse and model accurately.

In time series analysis, feature extraction helps us give the model more useful inputs. We can create new features from past values using lagged features, track recent trends with rolling averages, or capture repeating cycles with Fourier terms. In this example, we use a lag of one time step to show how earlier values can help the model learn patterns over time.

In this chart, we see how lagged features are created from the original time series. Lagged values represent earlier time steps and are added as new features to help the model learn from past behaviour. For example, “Lag 1” shifts the original data one step forward, allowing us to see how the previous value relates to the current one. This helps capture dependencies across time, which is essential in time series modelling. Ready to dig deeper? Use feature extraction to reveal what your time series data isn’t telling you at first glance.

Video 4 - Introduction to A R (Autoregressive) Models

If the past holds patterns, can we use them to predict the future? Autoregressive models do just that—by analysing previous data points to forecast what comes next. Let’s explore how this works and why it’s a fundamental skill for anyone working with time series data.

Let’s look at a simple example. Imagine you’re trying to estimate your electricity bill for this month using your bills from the past few months. That’s the basic idea behind an Autoregressive, or A R, model. It uses patterns in past data to predict future values. In time series analysis, A R models assume that what happens next depends on what has already happened—just like your next bill likely depends on your previous ones. In Autoregressive models, or AR models, we use past values to predict the current one. For example, an AR model of order p predicts the current value by using the last p values. The general formula is shown on the screen.

Here, Y_t is the value at time t , c is a constant, each ϕ_i represents a coefficient for a past value, and ϵ_t is the error term that captures random variation. Choosing the right order for an Autoregressive model is an important step. The value of p tells us how many past values we’re using to predict the current one. A higher p includes more history but can also make the model more complex. To find the best order, we use



tools like the Akaike Information Criterion, and the P A C F plot, which shows how strongly each past value is linked to the present.

Autoregressive models are used in many real-world situations. In finance, they help predict future stock prices using past performance. In weather forecasting, they use previous temperature or rainfall data to estimate future conditions. In retail, businesses use historical sales data to forecast future demand. These models work on the idea that patterns in the past can help predict what's coming next.

Here's a simple example of an Autoregressive model with order two, written as A-R-two. This means we use the last two values to predict the current one.

Let's say the value at time t minus one is ten, and the value at t minus two is eight. We apply the formula shown.

Y_t equals one point five plus zero point six times Y_{t-1} , plus zero point three times Y_{t-2} , plus epsilon- t .

Substituting the values gives us:

Y_t equals one point five plus zero point six times ten, plus zero point three times eight, which equals nine point nine.

This assumes no random noise, or that the error term is zero.

Autoregressive models can be useful, but they also have limitations. They only capture linear relationships, so they may miss more complex patterns. These models also assume that past trends will continue in the future, which isn't always the case. Finally, A-R models are sensitive to outliers—unexpected data points can distort predictions. Knowing these limitations helps us decide when an A-R model is appropriate and when other methods might be better.

Before you run your next prediction—ask yourself: is your data playing by the rules AR models expect? Spot the limits, challenge the assumptions, and choose your tools wisely.

Video 5 - Introduction to Moving Average (M-A) Models

Ever wondered how to make sense of unpredictable ups and downs in time series data? Moving Average models help us smooth out short-term fluctuations and focus on meaningful trends. They're a key tool for producing more stable and accurate forecasts. Let's learn how these models work and when to use them.

Let's begin with a simple idea. Imagine predicting your daily spending by averaging the past few days. This smooths out sudden changes and gives you a more stable estimate. In the same way, a Moving Average model uses past errors—random differences between predicted and actual values—to forecast future data points. It helps reduce short-term noise and reveals clearer trends in a time series.

A Moving Average model of order q predicts the current value by combining past error terms. Each error term is multiplied by a weight, called theta, which controls its influence on the prediction. The formula includes the mean of the series, past errors, and a final error term for time t . This approach helps us handle random shocks and smooth out unpredictable changes in the data.

The order q in a Moving Average model tells us how many past error terms are used to predict the current value. A higher q includes more of these past errors, which might improve accuracy—but it can also make the model more complex. To choose the right q , we can use tools like the Akaike Information Criterion, or AIC, and the Autocorrelation Function, or ACF. These help identify which past errors are most useful for prediction. Moving Average models are used in many real-world settings. In finance, they help smooth out stock price data, making it easier to spot trends. In weather forecasting, they reduce daily noise in temperature readings for more stable predictions. In retail or sales, MA models help reduce short-term demand fluctuations, giving businesses a clearer view of customer behaviour. Across fields, these models help focus on patterns, not



distractions.

In time series analysis, feature extraction helps us capture meaningful patterns from past data. Common techniques include creating lagged features, calculating rolling averages, or using Fourier terms to represent cycles and trends.

Let's look at a simple example of a Moving Average model of order two.

This model uses the two most recent error terms to predict the current value.

In this case, the past errors are minus one point two and zero point eight. The model formula is:

$Y_t = 2 + 0.5 \epsilon_{t-1} - 0.3 \epsilon_{t-2}$

Assuming no new error, or that $\epsilon_t = 0$, we substitute the values:

$Y_t = 2 + 0.5(-1.2) - 0.3(0.8)$

That gives us $2 - 0.6 - 0.24 = 1.16$.

While Moving Average models are useful, they do have limitations. They work best when the time series is stationary, meaning the data's properties stay consistent over time. MA models also rely on past error terms, which means they only capture short-term effects. If the data has complex trends or relationships not driven by errors, these models may not perform well.

Video 6 – Time Series – Demo

Hello and welcome everyone. In this demo, we are going to look into the time series concept. You might have gone through the concepts, but let us try to visualise them.

Let us try to connect our understanding with what actually happens. So, in this demo, we are going to load the dataset. We are going to plot the time series.

We are going to decompose that into different components and we are going to see them visually. Further, we are going to perform some basic types of modelling on this data and then try to understand that what are the shortcomings of each of these modelling and how do we evolve to the state of art models we have. To do so, we are going to implement our understanding on a very well-known dataset.

We call that as an Airline Passengers Dataset and we have this dataset on a monthly basis from 1949 to 1960. This dataset is available over this link and hence, I am using Pandas library to read this URL and when I do so, I see that these are the top 5 rows of my dataset starting from the 1st of January 1949, we have 112 passengers, 1st of February the same year 118 and so on and so forth. We are going to use this data for our analysis.

The first analysis is to plot the time series and visually see how it looks like. So, all what we need to do is we need to mention `plt.plot` and pass on this time series which is going to use these passengers and the month axis. So, you see the months are all listed on the x axis and the count of passengers on the y axis and if you see clearly this is the time series which looks linear because it is growing linearly, but there is a trend to this time series.

If you understand what trend means, if not we will see that. Now, if you remember and recall the definition of trend, seasonality etcetera, you know that trend is nothing, but the linear movement of your time series over a long period of time. And if you see the movement is



upwards, there is an either upward trend or a downward trend or no trend, but we see there is an upward trend.

Something like this you would see in the decomposed time series below. We also see there is some level of seasonality which is not linear because we see the seasonality component, which is very low over here, but we see a very high value over here. So, you would also see that.

To do so, Python libraries provide us very efficient ways of decomposing my time series. There are two levels of model there is multiplicative and additive model. What is multiplicative and additive? Multiplicative model assumes that my components have been multiplied together, additive assumes that my components have been added together.

Because I see a bit of non-linearity in seasonality over here, you see this range is smaller and larger over here. Based on this assumption, I select the multiplicative model. You can also try that with additive just to see what happens.

Well, when I select that multiplicative and I go by decomposing the plot, I see that this is what the outcome is. You see that this is my original time series, this is the trend of my time series, the trend which is decomposed from this time series and this is the seasonality component and then you have the residuals. It is the sum of these three what you see over here in this time series.

Now, let us try to, now you would argue saying that how do we access each of these components separately. So, you can do that by just plotting the trend, seasonality and residuals as a separate variable. So, this is your trend component, this is your seasonal component and this is your residual component and they are all done separately.

Now, you know that when you analyse a time series, if it has a trend, it is not a stationary time series, it is a non-stationary time series and before any analysis, we should convert this to stationary. You would see that in your live sessions, but for now let me move ahead and show you how do you build a basic naive forecasting model. Well, if I ask you given this is my history, what should be the next value? The most naive answer to this question would be my next value would be my previous value.

I would use this to build a baseline. So, what am I going to do is I am going to split my data into train and test. Remember, in time series when you split, you split it like this.

You take some last few values as test and the remaining values as train. You do not randomly shuffle and then you train and test split, do not do that. So, I take 134 observations in train to learn the pattern and implement that on the last 10 observations to see if it works well, it is a forecasting model.

Now, I basically say that my forecasted value is nothing but the last observed training value. So, I take the last training value and predict that for the next 10 steps. So, that is why we have the length of test.

So, these are all my because my last value was 391. So, my next 10 values are 391. Because of the basic assumption which we made over here, the next value is equal to the last observed value and that is what my data looks like.



If I go by plotting it, you would see the plot looks like this. You have the usual graph and then you have the time series. This is the actual value, but you see the red line dashed line that is the predicted value that is what the name forecast model has done.

Now, you see there is a clear difference between them, the actual and the predicted and that is very obvious because that is not how forecasting works. But as a baseline, we thought to start with that. We clearly see there is a difference, but if I want to quantify that difference, I am going to use a couple of metrics, the mean absolute error and the mean squared error.

I pass on the actual values and the forecasted value to mean absolute error and I do the same for squared error to get the root mean squared error value. So, my mean absolute error is 99.80, my root mean squared error is 123.63. So, in summary, it is a very simple method, it is good as a baseline and these are some of the reported metrics. Let us move ahead with a better strategy and we call that strategy as exponential smoothing.

What it does is, it is not going to give equal weightage to all the past values. In fact, it is going to decrease the weights exponentially of the past values. So, in a way, it is going to give more importance to the recent changes.

So, you see how it happens. We are going to use a smoothing level of 0.8. I want you to change these values and see what effect is on the output, but for now, I am going to take 0.8 and I am going to use this model to forecast for the next 10 months. When I plot the forecasted value, I get this as an output, which does not look very different from what we had earlier.

But, if I see the metric, so I see 96.22 and if you compare that with the earlier which was 99.80 and 123 and this is now 120. So, definitely I did improve slightly, but because your data has seasonality and your exponential smoothing algorithm is not very ideal for seasonality component of your time series, it does not show you a big improvement, but a significant small improvement is observed. So, we see that some of these models which work on basic assumptions do not tend to work very well.

And hence, we use some statistical modelling technique and the first statistical modelling technique is autoregressive model. You have seen in the slides that autoregressive model is nothing, but it is a combination, a linear combination of its past values. And the thing is that the coefficients of the equation is learned from the data itself.

This is a very famous equation. I say that my time series value is nothing, but some constant plus the coefficient of the previous value, another coefficient of the second last value so on and so forth, pth coefficient of the pth value last from the last and then some residuals which is what we call as a white noise. So, I am going to employ this equation and learn these coefficients from the data and then we are going to do predictions.

To do so, we employ this library AutoReg and I pass on my data. And I say that let us try to test on 5 previous values. Now, you might ask me why did I take 5? We will start with 5 and we can try with different values to see which one gives me the best result.

I fit my model and then I do the prediction on the test data. When I see the forecast, I do see that there is some improvement in its prediction, but the improvement is certainly not in the direction of the actual value. And probably the reason because I took away small value of p. And here I would suggest you to try with a larger value of p and see if this improves.



Now, quantifying it gives me a larger error because my time series is moving in the other direction. So, in a way if you see in terms of its route, it does improve better than stationary, but because it has a very strong seasonal data and the value of p actually decides a lot about forecasting, we see that it does not capture the seasonality component and hence it moves in the opposite direction. Similarly, we have another model called as a moving average model.

The explanation of that would have been there in the slides. If you want to see the implementation of that model, you can use this ARIMA library. More of this library you would see in the live session.

Here I take a q parameter. Again, I take the 5 past values. This is a number which you can change and I fit my model and I do the forecast on the next instance.

What I see is that this model completely moves in the opposite direction, but if you see it has very well mimicked the movement of your time series data although in the opposite direction, but it did try to mimic because the reason why it goes in the opposite direction, you see there is a bottom trend and that is something which is not followed. Had this been an upward trend, it would have actually followed in the upward direction. Again, the exercise which I would expect you to do is to change this to a higher value and see if it improves the prediction.

If I quantify that, I do get a larger value because it is completely in the opposite direction. So, you see that this is how we have evolved and in the live session you would see that how we would combine autoregressive and moving average to make our lives easier. Try to do that and compare the values against these values.

That would give you a realisation that how a problem in a real world is solved using a hybrid way of employing multiple algorithms and then forecasting. Thank you.

Video 7 - Time Series Data Preparation – Demo

Hello and welcome everyone. In this session, we are going to talk about a very critical concept and a very useful one also. The title is how do we prepare time series data for forecasting.

You know that time series data is temporal in nature. When I say it is temporal in nature, I mean to say that the previous value or the previous values of the time series are used to predict the future values. We have seen that in our slides.

So, we are going to see that what are the different measures keeping this in mind to treat our time series data. And we may or may not use all of them every time, but for different kind of problems, we will tend to use majorly these different types of preprocessing the time series data. We are going to use the same data set which we will use across our analysis, across our demonstrations about time series data.

So, let's start with the first task that is something called as date-time indexing. So, the date-time indexing is nothing but the objective here is to take your column, a specific column which you believe is a date-time type column and convert that to a proper date-time type column in Python. And then set that as the index, because you know that if I define time series, time series is something which is any observation, any phenomena which is captured at regular time interval.



So, my time series is actually characterised by time and that's the reason why I will have to track it against a time unit. So, whichever column we see is the time unit, we use that column, set it as the index and use that for processing. Well, let's read the data set first.

This is my URL and I read the CSV file, the URL, and this is how my output looks like. You see that we have the month and then we have the corresponding passengers for that month. You can visit this website, the GitHub link to actually see the entire data.

Now, on this data, you can clearly see that this is your date-time type, but this doesn't look in a very standard format. It says 1-1949, 2-1949, which eventually means that it is the month of January, the month of February, the month of March, but it doesn't tell you the date. And you might argue saying that we don't need the date.

Well, we would need the dates. For example, I want to convert this time series from a monthly frequency to daily frequency. I would need to understand whether this is the 15th day count or the first day count or the 30th day count.

So that is the reason why we should always try to use date-time indexing to index the corresponding column with the right data type. How do we do that? It's very simple, very straightforward. I'm going to use Pandas for that.

And I'm going to say 2-date-time. When I say 2-date-time, I mean to say that I'm going to convert that to a date-time object and whichever column I want to convert, I'm going to pass that. And the moment I do this, you will see that it will convert it to a date-time type.

And not only that, it also tells you that the frequency with which it is, you can clearly see it's monthly. And then the name of the column is month. And now you can set that as an index.

Once you have converted that to date-time, then you set that as an index. Now if you see here, after indexing how it looks like, it's more now in the appropriate format. So that's the very first thing which we should always try to do.

Now the advantage of indexing is that you can actually plot that in a very simple, you can simply say `df.plot`. You don't have to say anything and mention the title and you will see it will actually plot you using your x value that is your index. So a simple PD2 date-time is a simple way to convert any date-time object to the proper format. And you see it has taken the first of every month as the date which was used to record.

If you want, you can choose a different date also. But ideally, we'll go with the first one. The next step, once we have indexed our time series data, the next is to check and handle the missing timestamps.

Here, we are actually going to see that do we have any missing months, missing days, or missing weeks, whatever is the interval of my time series, we're going to check that. Because you would want our time series to be continuous in nature. If there is any sort of discontinuity, it will not work.

Now to do that, let us try to take the start date and the end date. The start date, that is the minimum date, and the end date and generate all the MS means on a monthly basis. The month start from that.



Start means the first date and M means the month. From this date, the minimum to the maximum generate me all the possible frequencies. See, the moment I run this, I will actually get the entire full range of data.

Now this becomes your ground truth or the gold standard. And this is the range which you can compare your time series with. If your time series range matches with this range, it means there are no missing time range, timestamps.

Now, because I have created a full range, I'm going to re-index my data frame with that full range. I'm re-indexing that with the full range. And I'm naming that index as month.

Now, I have forced my data frame to align with these new full data date range. If there are missing values, then you'll be able to now check that easily. So before you check a missing value, align, forcefully align that with the full possible range of timestamps.

That's what we did. Now you can simply do a check. I can say `isna.sum`. Now in the passengers column, I don't see.

See, there are no zero, no missing values. So ideally I say that there are no missing values in the timestamps. It means that all these ranges have the corresponding value.

All these timestamps have a corresponding passenger value and hence I don't see zero or nine over there. It says there are no nine values. If there had been a missing value, usually what we do is you tend to replace that with mean values, etc.

But in time series, there are different ways of filling. I have shown the options here. For example, if you want to fill that with the last known value.

For example, if you find out that the first of 1950, the first month of 1950, the value is missing, you will take the last known value. If you know the value of 1949 December, you'll take that. 1949 November, you will take that.

So whatever is the last known value, we are going to take that. And it is done by simply saying `fillna` and method is `ffill`, which is forward fill. Use the previous value and forward that to fill the next value.

And because we don't have any missing values, so it won't have any effect on that. But if you want, you can forcefully take a missing value and try to apply this method. The next one is the backward fill, which is nothing but the one which is the next value.

I am going to use that to fill the previous one. Now, there is one more method which is called as interpolation. And it is so simple.

You can simply say take that column dot interpolate by a linear method. So I will use some past values to interpolate the next missing value. So these are three different methods.

The one to take the last known value, the one to take the next known value and the one which takes some previous values and linearly tries to formulate the next value. Now, the question is which one works the best. It will depend upon the problem statement, the domain knowledge that which one you think is the best value.



Or you can try one of these three and see or all of these three and see which one gives you the better result. Well, let's move ahead to the next task, which talks about creating something called as time based features. You know that in time series, my time axis is the feature.

I did tell you that my time series is defined by my timestamps. The main timestamps could also be disintegrated into month, year, dates, etc. How do we do that? Because you have already converted into the date time type, this becomes very easy.

For example, if you want to capture the year, you can simply say `df.year` and `df.index.month`. For month, month. For quarter, for the day of the week, for the name of the month.

So you see, there are so many ways in which you can extract the components of the time index. And this is the advantage of using a date time type object. This is only possible with a date time object.

Now when you run this, you see for each month, what have I done is I have extracted its year, its month, its quarter, its day of the week and the name of the month. Now you can actually use these as features also, like which quarter it was, which day of the week was, for you to know that what were the number of passengers which had purchased the ticket to fly with that airline that day. It is very important to know that which week, which day of the week it was.

If it was weekend, weekday probably more people, weekdays less people, weekends more people. So that would have been the case. So that kind of analysis is very much possible if you are able to extract the different components of your time.

And then at least what you can do is you can try to study month wise the distributions. You clearly see that it says that in the month of July, you see that there is a distribution in the month of August, the distribution is wide and you see, you get to see more people travelling compared to other months like November and February where comparatively less people travel. That is a very great analysis to observe.

And this is what we call as a box plot, you might have done this in exploratory data analysis. This is just the month wise distribution plot. Well, the next task is there is something called as lag features.

Now you know that your time series is temporal in nature, that is your time series will use the past values to predict the next one. We call that past values as lag features. Now how much lag I want depends upon how much of the past values I want to use.

See $t - k$, where k is a lag interval. If you say one month ago, it will use the previous value. Two months ago, it will use the two previous values.

Twelve months ago, it will use the twelve peak in previous values. And this is very important in the time series analysis domain. Let me show you that.

So, you take your percentage of data and you create a lag feature by shift. Shifting by 1 means you are taking a lag of 1, by 2 means by taking a lag of 2, by 3 means taking a lag of 3, by 12 means by taking a lag of 12. So, now you see that.

What is happening is for this month, this was the value. Now, what was the previous value? Well, this was the first value. So, there was no previous value.



So, all of these three would be nines. For the second month, what was the previous value? See, this is the previous value. See, current value is 118.

This previous value is now considered as lag 1. There was no third previous value. So, nine, twelfth previous value. So, nine.

For this month, March, my value is 132. But what was my previous value? 118. Third previous value, nine.

Twelfth previous value, obviously nine. Then month fourth, 129, my previous value is 132. So, you see how corresponding to each value, the previous value is changing.

And now for the first time, we will see that there is a 3 month ago value. See, 3 month ago. 1, 2, 3. So, that 3 month ago 112 value will appear over here.

This will still be nine. So on and so forth, I am creating lags, you see, for every value. This is the previous value lag.

This is the third previous value. This is the twelfth previous value. This is the current value, the previous value, the third previous value, and the twelfth previous value now appears.

So, if I want to predict this right, I would need the previous value to predict. I would need the previous to previous to previous value to predict. I would need the twelfth previous value to predict, isn't it? I would need these past values to predict.

That's what we call as features. And because these values are its own previous values, we call that as lag features. Now, the question is that which of the previous values should you take? Well, we will answer that when we do modelling.

But for now, this is how you create lag features. Now, you know that when you do so, it is creating nan values. So, you need to drop those nan values.

Simply drop by saying drop na. And this is what you get as a final correlation, which tells you that what is the correlation of the value with its past values. It is saying that for passenger's data, the correlation with its lag 1 value is 95, which is massive, lag 3 is 81, lag 12 is 99.

It is a very massive correlation of the passenger's data with its lagged value. So now, you can do this. You have done for the first one, the third one, the twelfth one.

If you want to do for all of them, from 1 to 12, you can do it like this. And you can actually get all the lags, all the previous lags. The next task is to create something called as rolling features.

Now, in rolling features, what we do is we try to observe that if my time series has unusual fluctuations and how can we smoothen them out. Now, if I want to smoothen them, and I want to use the past 3 values to smoothen them, I want to calculate something called as a 3-month rolling average. What do you, what would you observe is, how do I do that? Take the passenger's column and use something called as a dot rolling and tell the window that how many past values do you want to use to calculate the mean.

Rolling average means how many past values you want to use to calculate mean. When I say rolling window is 3, I am going to use 3 past values to calculate the mean. So, you see we have rolling mean 3, we have rolling mean 6 and rolling mean 12.



The question is that how did we decide 3, 6 and 12? Because that is basically something which comes from experience. When you try to solve more and more time series problems, you will eventually learn that which of these rolling averages will work for your own problem. There is no clear indication about what should be the average.

Usually, I tend to use 3 or 4 or 5 maximum. But you can have even 6 and 12. When you use 6, it is going to capture half yearly.

When you use 12, it is going to capture yearly. For 3, it is going to be quarterly. For 1 and 2, it is going to be just, 1 does not make sense, at least 2. But taking an average of 2 is not going to help you a lot.

So, once I am done with that, you see for each passenger, I have the rolling mean, the average of the past 3 values, past 6 values, past 12 values. Now, because it is a rolling mean and again, you do not have a past value to it, it will give you 9. Likewise, the previous one. Now, as I have this result, you see that I go by plotting it and this is a classic plot of rolling averages, a 3-month and a 12-month.

You see how a 12-month is capturing the trend of the time series, how a 3-month is capturing the seasonality of the time series. That is actually very good. So, you know that it is going to create 9 values.

So, be very cautious about it. Thank you everyone.